IEEE Open Journal of the Computer Society

Received 2 April 2026; revised 23 May 2026; accepted 25 May 2026. Date of publication 2 June 2026; date of current version 11 June 2026. The review of this article was arranged by Associate Editor Bahman Javadi.

Digital Object Identifier 10.1109/OJCS.2026.3699545

Idle Fragmentation-Aware Resource Scheduling for Hyperscale Energy-Efficient Cloud Data Centers

B.M. BEENA [1] (Senior Member, IEEE), JIYA BORIKAR[1], LOHITHA KANISETTYPALLI [1], HARSDA SHRIVASTAVA [1], AND RAMESH RAMASWAMY [2]

[1]Department of Computer Science and Engineering, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Bengaluru 560035, India
[2]School of Management, University at Buffalo, Buffalo, NY 14260-4000 USA

CORRESPONDING AUTHOR: B.M. BEENA (e-mail: bm_beena@blr.amrita.edu).

ABSTRACT Hyperscale cloud data centers are rapidly expanding to support the growing computational demands of AI/ML workloads. This growth increases power density, where even small inefficiencies result in significant energy consumption and cooling overhead. A key challenge in such environments is idle CPU–memory fragmentation, where scattered and unusable resource portions prevent efficient workload placement. As a result, underutilized machines remain active, leading to unnecessary energy usage. To address this problem, this study proposes an Idle Fragmentation-Aware Scheduling (FAS) framework. The framework introduces fragmentation-aware resource modeling and an intelligent scheduling mechanism to reduce residual CPU–memory imbalance at both node and cluster levels. Workload characteristics and fragmentation patterns are analyzed to design effective multi-resource scheduling policies that improve packing efficiency.The proposed FAS framework is implemented and evaluated in a Kubernetes environment, where container workloads are scheduled based on real-time fragmentation conditions. An energy-aware consolidation strategy is applied to safely shut down underutilized nodes. Experimental results show that FAS reduces the Resource Fragmentation Index (RFI) by 33.14%, improves average utilization by 24.8%, and decreases energy consumption by 36.8%, while maintaining 100% job acceptance and SLA reliability. These results demonstrate that fragmentation-aware scheduling significantly improves resource utilization and energy efficiency. Overall, the proposed approach provides a practical solution for sustainable cloud management and supports energy-aware orchestration aligned with UN SDG 7 (Affordable and Clean Energy) and SDG 12 (Responsible Consumption and Production).

INDEX TERMS Energy efficient scheduling, kubernetes orchestration, resource fragmentation, workload consolidation, docker, cloud automation, green computing, sustainable computing.

NOMENCLATURE

RFI	Resource Fragmentation Index used to quantify CPU memory imbalance.
SLA	Service Level Agreement.
$f_{\text{cpu}}, f_{\text{mem}}$	Fractional free CPU and memory after placement.
C_k, M_k	Total CPU and memory capacity of node k.
c_i, m_i	CPU and memory demand of job/pod i.
u	Utilization level of a node.
$P(u)$	Power consumption at utilization u.
P_{idle}	Idle power consumption of a node.
P_{max}	Maximum power consumption of a node.
$\mathcal{M}$	Set of machines/nodes in the cluster.
j_i	Job/pod i represented as (c_i, m_i, d_i).
d_i	Execution duration of job/pod i.
is_short(j)	Job duration classifier (short vs. long).
R	Number of consolidation iterations.
$\text{Freed}_{\text{Count}}$	Number of nodes freed (powered off).
$S_k(j_i)$	Fragmentation-aware placement score of job j_i on node k.
$\text{Score}_{\text{min}}$	Minimum fragmentation-aware placement score used to select the best node.
RFI_k	RFI of node k after placement.
$\text{RFI}_{\text{after}}$	RFI of a node after placing a job.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

MigrationOps Total number of workload migration operations performed during consolidation.

I. INTRODUCTION

Hyperscale datacenters have become the backbone of modern digital ecosystems, supporting applications such as artificial intelligence, e-commerce, healthcare analytics, microservices, and high-performance computing. As workload intensity continues to grow, improving resource utilization while minimizing energy consumption has become a critical challenge. Even at low utilization levels, servers consume a significant fraction of their peak power, leading to increased operational costs and higher carbon emissions.

A key reason for this inefficiency is idle CPU–memory fragmentation, where scattered and imbalanced resources remain unusable for incoming workloads. Such fragmentation prevents effective container placement, forcing lightly loaded nodes to remain active. This problem is particularly critical in containerized environments such as Kubernetes, where fine-grained allocation creates residual resource imbalance.

Traditional approaches to improving energy efficiency primarily rely on virtual machine consolidation. For example, trend-based consolidation techniques leverage workload prediction to reduce unnecessary migrations and energy usage [1]. However, with the increasing adoption of containerized architectures, Kubernetes-based scheduling has become a central research focus. Surveys on Kubernetes schedulers highlight several limitations, including resource request–usage mismatch, lack of energy-aware mechanisms, and absence of explicit handling of CPU–memory fragmentation [2]. Rao et al. [3] proposed an SLA-driven scheduling mechanism for microservices to reduce energy consumption. Similarly, energy-aware elastic scaling approaches dynamically adjust resource allocation to minimize idle capacity and improve utilization [4]. Studies have also demonstrated that the design of Kubernetes schedulers significantly impacts overall cluster energy consumption [5].

In addition, dynamic scheduling frameworks such as KubeDSM incorporate migration-aware placement strategies to improve resource utilization across distributed environments [6]. Hybrid VM–container consolidation approaches further attempt to bridge traditional and containerized resource management techniques [7]. More recently, carbon-conscious scheduling frameworks have been introduced to incorporate environmental factors such as carbon intensity into workload placement decisions [8].

Despite these advancements, existing approaches primarily focus on high-level optimization objectives such as energy, performance, or carbon efficiency, without explicitly modeling fine-grained CPU–memory fragmentation. Therefore, the key research gap lies in the absence of a fragmentation-aware Kubernetes scheduling mechanism that can quantify CPU–memory imbalance and guide placement decisions based on the geometry of remaining resources.

To address this problem, this paper proposes a Fragmentation-Aware Scheduling (FAS) framework. The proposed approach introduces a Resource Fragmentation Index (RFI) to quantify imbalance between remaining CPU and memory resources. A fragmentation-aware scoring mechanism is used to guide container placement, and an energy-aware consolidation strategy is applied to enable safe node shutdown without violating SLA constraints. Unlike prior VM-level or simulation-driven methods, FAS is implemented and validated in a Kubernetes environment using Alibaba production trace workloads, demonstrating practical applicability and scalability.

The main contributions of this work are as follows:

- A fragmentation-aware scheduling framework that models and minimizes idle CPU–memory imbalance at both node and cluster levels.
- A multi-resource placement strategy that extends Best-Fit heuristics with workload classification and fragmentation-aware decision logic.
- A Kubernetes-based evaluation using real Alibaba Cluster Trace workloads, ensuring realistic and scalable validation.
- An energy-aware consolidation mechanism that enables safe node shutdown and reduces overall power consumption.
- A comprehensive experimental evaluation demonstrating improvements in utilization, fragmentation reduction, and energy efficiency over baseline schedulers.

The remainder of this paper is organized as follows: Section II presents a detailed review of existing work related to scheduling, consolidation, and fragmentation-aware optimization in cloud environments. Section III describes the dataset used in this study, outlines the problem formulation, presents the proposed Fragmentation-Aware Scheduling (FAS) methodology. Section IV explains the experimental setup and implementation details within a Kubernetes cluster. Section V discusses the experimental results and analysis, including fragmentation trends, utilization improvements, and energy savings. Finally, Section VI concludes the paper and highlights potential directions for future research in fragmentation aware and energy efficient cloud scheduling.

II. RELATED WORK

Energy-aware scheduling in large-scale cloud infrastructures has evolved significantly with the increasing complexity of workloads and the transition toward containerized deployments. Early resource management approaches primarily focused on virtual machine (VM)-level consolidation [1], [9], [10], gradually moving from simple heuristics to more intelligent and context-aware strategies. For example, EEVMC proposed by Rehman et al. [11] integrates overload detection, underload identification, and efficient VM selection to improve consolidation and reduce energy consumption. Complementing these approaches, thermal-aware scheduling frameworks were introduced to jointly optimize computational performance and cooling efficiency. Mao et al. [12] incorporated thermal signals into scheduling decisions, while Lin et al. [13] proposed a holistic thermal energy

optimization model that combines IT load balancing with inlet temperature awareness. Although these methods significantly improve energy efficiency, they operate at coarse VM granularity and do not capture fine-grained CPU–memory fragmentation in modern containerized environments.

With the increasing diversity and dynamism of cloud workloads, machine learning and reinforcement learning (RL)-based scheduling approaches have gained significant attention due to their ability to adapt to complex environments. DeepBS proposed by Li et al. [14] models cloud scheduling as a Markov decision process to optimize resource allocation under QoS constraints. Similarly, PPO-based approaches [15], [16] have been employed to improve energy efficiency and performance in heterogeneous and cloud–fog environments. These methods demonstrate improvements in throughput, latency, and energy consumption by learning adaptive scheduling policies. However, most RL-based approaches are evaluated in simulation-based environments and do not fully capture real-world challenges such as resource fragmentation caused by container requests and limits. Furthermore, their effectiveness depends on accurate energy feedback, which remains difficult to obtain in practice. Centofanti et al. [17] highlight this limitation by showing that existing container-level power measurement tools exhibit significant variability and lack consistency for reliable scheduling decisions.

Parallel research has explored containerized resource management more directly, focusing on consolidation strategies, resource balancing, and sustainability. Hybrid VM–container consolidation approaches, such as the method proposed by Kiaee and Arianyan [7], combine deep learning-based feature extraction with multi-criteria placement strategies to improve decision-making in edge–cloud environments. Heuristic-based schedulers such as BalCon [18] further enhance consolidation by dynamically adapting placement strategies to reduce migration overhead under fluctuating workloads. In addition, sustainability-driven scheduling frameworks [19], [20], [21], [22] incorporate carbon intensity signals, renewable energy awareness, and performance-aware resource allocation into scheduling decisions to improve environmental efficiency. While these approaches improve consolidation quality and sustainability, they primarily optimize high-level performance and energy metrics and do not explicitly model the impact of fine-grained CPU–memory fragmentation on future workload placement and consolidation efficiency.

Recent advances in cloud scheduling have increasingly focused on Kubernetes-based orchestration, reflecting the shift toward containerized microservice architectures. Multi-criteria schedulers such as KCSS proposed by Menouer [23] utilize decision-making techniques that jointly consider CPU, memory, storage, power consumption, and container-related metrics to improve node selection and system stability. Similarly, hybrid scheduling strategies in heterogeneous environments [24] integrate intelligent workload pairing and optimization techniques to improve scheduling efficiency under dynamic workload dependencies. In addition, container-focused studies and systematic reviews [2], [25], [26], [27] highlight the importance of Kubernetes-native scheduling, real-world deployment constraints, and the need for improved resource allocation strategies in distributed environments. Despite these advancements, existing Kubernetes schedulers largely rely on aggregate resource metrics and lack explicit mechanisms to quantify and minimize CPU–memory fragmentation at the container level.

More recent research has focused specifically on energy-aware scheduling in Kubernetes environments, highlighting the growing importance of efficient resource utilization at the container level. Rao et al. [3] proposed an SLA-aware scheduling strategy that incorporates communication patterns and resource utilization to reduce cluster energy consumption while maintaining QoS guarantees. Energy-aware elastic scaling approaches [4] further improve efficiency by dynamically adjusting resource allocation to minimize idle capacity and enhance utilization under fluctuating workloads. In addition, experimental evaluation of heuristic Kubernetes schedulers by Daimari et al. [5] demonstrates that scheduler selection itself can significantly influence overall energy consumption, emphasizing the importance of intelligent scheduling policies in sustainable cloud environments. Carbon-conscious scheduling frameworks [8] extend this direction by incorporating environmental signals such as carbon intensity into workload placement decisions to reduce emissions while maintaining service-level performance. While these approaches advance energy-efficient cloud operation, they primarily focus on global optimization objectives and do not explicitly address fine-grained CPU–memory fragmentation in scheduling decisions.

Dynamic and adaptive scheduling frameworks have further advanced resource management in distributed and containerized environments. KubeDSM [6] introduces a Kubernetes-based dynamic scheduling and migration framework that improves resource utilization in cloud-assisted edge clusters through migration-aware placement strategies. Similarly, performance- and energy-aware scheduling mechanisms [21] jointly optimize workload placement and energy consumption by incorporating system-level performance constraints and real-time workload behavior. Emerging AI-driven autoscaling solutions such as NeuroScaler [28] further enhance this direction by dynamically adjusting resource provisioning for container-based services based on workload demand patterns. Although these adaptive approaches improve scalability and responsiveness, they primarily focus on migration and autoscaling efficiency and do not explicitly model residual CPU–memory fragmentation or its long-term impact on fine-grained scheduling decisions.

Additional research has explored advanced optimization and workload-aware scheduling strategies in cloud and containerized environments. Workflow-aware resource allocation methods such as CWRA [27] leverage task prediction techniques to dynamically scale resources and improve utilization efficiency during runtime execution. Stochastic

bin-packing formulations [29] address uncertainty in container placement using probabilistic constraints, enabling more robust scheduling under dynamic workload conditions. Multi-objective optimization approaches [30], [31], [32] further demonstrate reductions in power consumption and carbon footprint through predictive resource allocation mechanisms. In addition, comprehensive surveys on energy-aware server consolidation [33] provide structured taxonomies of VM placement, migration, and consolidation strategies, identifying persistent challenges related to workload imbalance and underutilized infrastructure resources. Theoretical studies on parallel performance and scalability [34] further highlight the trade-offs between resource scaling efficiency and diminishing computational returns, which directly influence scheduling efficiency in distributed cloud environments. Complementary studies on network-aware VM placement [35] provide additional insights into communication-aware scheduling decisions that influence resource allocation efficiency across distributed clusters.

Recent research demonstrates the growing adoption of cloud computing architectures to enable scalable, intelligent, and sustainability-oriented systems across diverse application domains. Nayini et al. [36] proposed a cloud-based Health Information Exchange platform deployed on AWS to support secure healthcare data sharing and disease prediction using integrated machine learning services. Beena et al. [37] addressed operational and environmental challenges in multi-cloud environments by introducing a unified self-service orchestration portal with custom cloud connectors across AWS, Azure, and Google Cloud, incorporating sustainability-aware resource management strategies to improve utilization and reduce carbon footprint. Cloud-based intelligent media processing has also been explored by Kumar et al. [38], who presented a cloud-enabled image processing and augmentation framework supporting scalable and distributed data workflows. These works highlight the importance of scalable cloud infrastructure across application domains; however, they focus primarily on application-level orchestration rather than low-level scheduling efficiency, leaving CPU–memory fragmentation challenges within container schedulers largely unresolved.

Overall, modern scheduling strategies demonstrate strong advances in energy reduction, consolidation quality, thermal awareness, sustainability, and adaptive decision making. However, most existing techniques either operate at the VM level, rely on simulated environments, or focus on macro level metrics such as cost, delay, or carbon intensity while fine grained resource fragmentation remains largely overlooked. The literature lacks (i) explicit fragmentation modeling in containerized clusters, (ii) schedulers that directly target CPU memory fragment minimization, and (iii) real Kubernetes based validation combining fragmentation-sensitive placement with power aware node consolidation. This work introduces a Fragmentation-Aware Scheduler (FAS) that systematically measures and reduces fragmented idle CPU memory resources using trace driven workloads, integrates fragmentation aware multi resource placement into Kubernetes, and applies energy aware node shutdown to achieve meaningful reductions in cluster level energy consumption.

TABLE 1. Comparison of Existing Scheduling Strategies With the Proposed FAS

Existing Works	Proposed FAS
VM-level consolidation with limited container-level awareness [1], [9]–[11]	Container-level scheduling with explicit CPU–memory fragmentation modeling
Energy reduction achieved indirectly through consolidation or thermal-aware placement [12], [13], [19], [20]	Direct quantitative energy reduction (36.86%) through fragmentation-aware consolidation and safe node shutdown
Most Kubernetes schedulers rely on aggregate resource metrics without explicit residual imbalance modeling [2], [23], [25], [27]	Computes Resource Fragmentation Index (RFI) to quantify residual CPU–memory imbalance
Limited validation of fragmentation-aware consolidation in real Kubernetes environments [4], [6], [7], [28]	Validated using Alibaba cluster trace dataset and real Kubernetes deployment [39]
Existing works improve utilization or scaling but do not establish explicit fragmentation–energy correlation [26], [32]–[35]	Demonstrates numerical relationship: ↓RFI ⇒ ↑Utilization ⇒ ↓Energy Consumption

TABLE 2. Summary of Trace Files Used

File Name	Description
server_usage.csv	CPU memory usage of each machine
container_event.csv	Container creation and termination logs
container_usage.csv	Actual CPU memory usage of running containers
batch_instance.csv	Execution details of batch workloads

III. METHODOLOGY

The proposed Idle Fragmentation-Aware Scheduling (FAS) framework integrates intelligent scheduling automation, energy-efficient resource management, and fragmentation-aware design principles to detect, model, and minimize scattered CPU memory fragments in large scale containerized cloud environments. The complete workflow consists of six phases, beginning with Alibaba trace preprocessing and culminating in Kubernetes-based validation and energy-aware node management. Fig. 1 illustrates the overall architecture and workflow of the proposed system.

A. DATA DESCRIPTION

This study uses the Alibaba Cluster Trace dataset [39], a publicly available large scale production workload trace widely adopted in cloud scheduling research. The dataset contains real CPU and memory usage patterns, container lifecycle events, and machine level resource statistics collected from thousands of servers. For our experiments, we extract job resource requests, actual workload utilization, and node usage snapshots to recreate realistic scheduling conditions inside Kubernetes. Table 2 summarizes the primary trace files utilized.

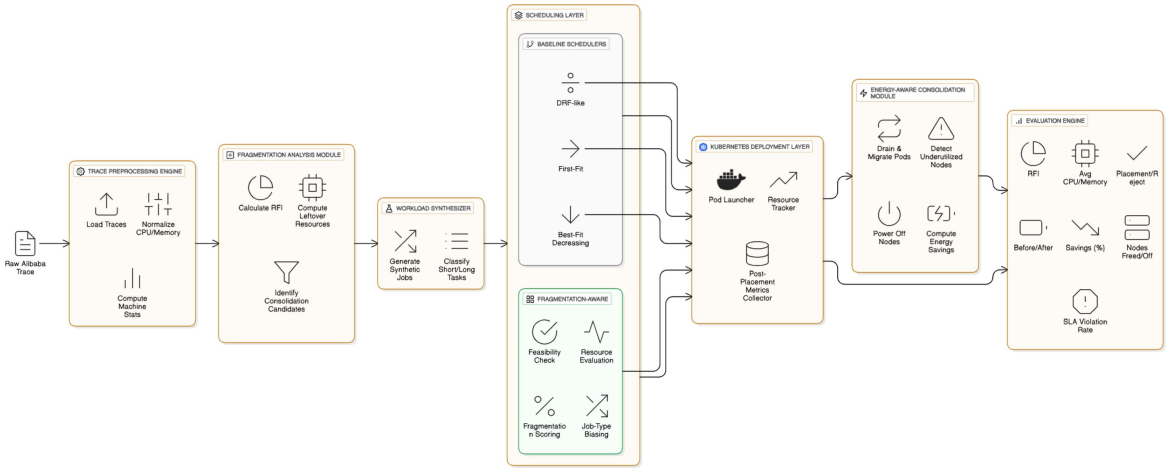


FIGURE 1. Overall system architecture and workflow of the proposed fragmentation-aware scheduling framework.

B. INTEGRATION OF FAS

The Alibaba Cluster Trace dataset is preprocessed to generate machine-level resource statistics essential for fragmentation analysis. Server usage logs are merged with machine capacity records to obtain valid CPU and memory capacities for each node, and CPU usage percentages are converted into core units for consistency. Mean CPU and memory utilization are computed per machine, after which leftover resources are derived by subtracting actual usage from capacity and then normalized to enable fair comparison between CPU and memory availability. These normalized idle resources form the basis for computing the Resource Fragmentation Index (RFI). Idle resource fragmentation refers to the presence of unused CPU and memory resources that remain unavailable for future workload placement due to imbalance between resource types. The RFI quantifies how scattered and unusable the remaining resources are on each machine. Batch task logs are further analyzed to extract representative CPU and memory request patterns, enabling estimation of how many typical tasks can fit into each node's idle capacity. Machines exhibiting low utilization but high RFI are finally identified as consolidation candidates, forming the processed dataset used for subsequent scheduling and consolidation stages.

C. BASELINE SCHEDULING

A baseline scheduling system is first constructed using the First-Fit (FF) strategy to quantify cluster behavior without fragmentation-aware optimization. Machine level statistics from the preprocessed dataset are loaded, and a fixed number of representative tasks based on the 75th percentile CPU and memory request are sequentially submitted. Each task is placed on the earliest machine that has sufficient remaining CPU and memory, after which the machine's residual resources are updated. Once all tasks are processed, post-placement CPU and memory gaps are used to recompute the Resource Fragmentation Index (RFI), revealing the degree of unused yet unusable capacity created by FF placement.

TABLE 3. Characteristics of the Schedulers Implemented in This Study

Scheduler	Advantages	Limitations
FF	Fast and simple baseline scheduler	High fragmentation; poor packing quality
BFD	Produces tighter packing than FF	Higher computational overhead for large clusters
DRF	Ensures fairness across CPU memory resources	Weak consolidation; lower energy efficiency
FAS	Lower fragmentation; higher utilization; better consolidation	Requires FI computation in preprocessing

The cluster's total power consumption is then estimated using the linear utilization-based model shown in (1):

$$P(u) = P_{\text{idle}} + \left(P_{\max} - P_{\text{idle}}\right) u \tag{1}$$

and compared against an ideal scenario where the same workload is packed onto the minimum number of machines capable of accommodating the total resource demand. This analysis yields the baseline fragmentation level, job placement success rate, and the amount of energy wasted due to inefficient spreading of tasks across nodes. Although a linear utilization-based power model is adopted for simplicity and reproducibility, it does not capture non-linear thermal dynamics observed in hyperscale cooling systems. Future work will incorporate non-linear thermal-aware power models to better represent realistic energy behavior.

D. PROPOSED FRAGMENTATION-AWARE SCHEDULER

Table 3 presents a comparison of the four schedulers evaluated in this study, highlighting their respective strengths and limitations. These baseline characteristics motivate the need for a fragmentation-aware scheduling strategy capable of improving packing quality and enhancing energy efficiency.

The evaluation compares three traditional schedulers—First-Fit (FF), Best-Fit Decreasing (BFD), and a DRF-like dominant share policy with the proposed Fragmentation-Aware Scheduler (FAS). A synthetic job set is generated using lognormal sampling of CPU and memory demands, and job

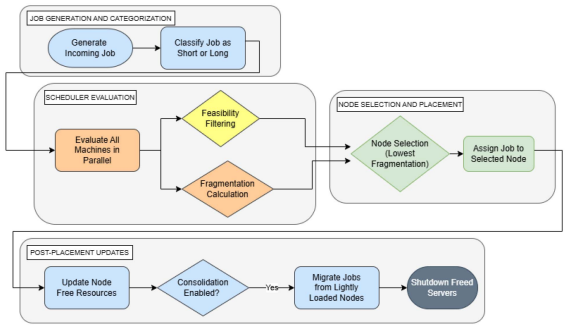


FIGURE 2. Idle Fragmentation-Aware Scheduling (FAS) task scheduling model and system workflow.

durations are used to classify workloads into short and long tasks.FF assigns each job to the first machine that satisfies its CPU memory requirements, while DRF selects the node that minimizes the resulting dominant share. BFD chooses the machine that provides the tightest post-placement fit based on leftover CPU memory fractions. FAS improves upon these policies by selecting the node expected to yield the least fragmentation. Its scoring function incorporates fractional leftover CPU and memory and applies a job-type bias—packing short jobs into smaller gaps and reserving larger regions for long tasks. After placement, optional consolidation migrates jobs from lightly loaded machines to free entire nodes.All schedulers are evaluated using the post-placement Resource Fragmentation Index (RFI), job acceptance rate, and energy consumption computed via the utilization based power model. Results show that FAS consistently reduces fragmentation, improves placement efficiency, and enables more nodes to be powered off safely.

The core decision logic of the proposed scheduler is summarized in Algorithm 1, and its consolidation stage is implemented through the migration procedure in Algorithm 2; together with the task scheduling model illustrated in Fig. 2, these components form the complete Fragmentation-Aware Scheduling workflow.

Algorithm 1: Fragmentation-Aware Scheduler (FAS).

Require: Job $j = (\text{cpu}_j, \text{mem}_j, \text{is_short})$, Set of Machines $\mathcal{M}$
Ensure: Allocation success (Boolean)
Pre-Sort: Jobs are sorted by $(\neg\text{is_short}, \text{cpu}_j + \text{mem}_j)$ in descending order.
$M_{\text{best}} \leftarrow \text{null}$, $\text{Score}_{\min} \leftarrow \infty$, $\epsilon \leftarrow 10^{-12}$
for each machine $M_i \in \mathcal{M}$ **do**
  if M_i.can_fit(j) **then**
  {Compute post-placement free-resource fractions}
    $f_c \leftarrow \frac{M_i.\text{cpu}_{\text{left}} - \text{cpu}_j}{M_i.\text{cpu}_{\text{cap}}}$
    $f_m \leftarrow \frac{M_i.\text{mem}_{\text{left}} - \text{mem}_j}{M_i.\text{mem}_{\text{cap}}}$
    $\text{Leftover}_{\text{total}} \leftarrow f_c + f_m$
  {Compute post-placement Resource Fragmentation Index (RFI)}
    $\text{RFI}_{\text{after}} \leftarrow 1 - \left(\frac{\min(f_c, f_m)}{\text{Leftover}_{\text{total}} + \epsilon}\right)$
  {Determine weight: 2.0 for short jobs, 0.5 for long jobs}
    if j.is_short **then**
      $w \leftarrow 2.0$
    else
      $w \leftarrow 0.5$
    end if
    $\text{Score} \leftarrow \text{RFI}_{\text{after}} + w \times \text{Leftover}_{\text{total}}$
    if $\text{Score} < \text{Score}_{\min}$ **then**
      $\text{Score}_{\min} \leftarrow \text{Score},$; $M_{\text{best}} \leftarrow M_i$
    end if
  end if
end for
if $M_{\text{best}} \neq \text{null}$ **then**
  M_{best}.place(j)
  return True
else
  return False
end if

E. CONTAINERIZED SIMULATION (DOCKER + KUBERNETES)

The fragmentation-aware scheduling workflow is extended to a live Kubernetes environment by deploying the pod workloads produced in the earlier stage. The allocation file is parsed to obtain the CPU and memory usage of each job in normalized core and GiB units. Kubernetes node capacities are retrieved from the API, and effective allocatable resources are computed after reserving small CPU and memory headrooms to ensure that system daemons and background Kubernetes processes remain stable.

Workloads are sorted in decreasing dominant-resource demand so that larger jobs are placed earlier. A minimal active node set is initialized using the theoretical lower bound on required nodes, computed from the ratio of total workload demand to effective per-node capacity under a target utilization constraint ($u_{\max} = 0.92$). Each incoming job is evaluated against the active nodes using a custom placement score that incorporates post-placement dominant share, residual free CPU memory balance, and remaining fill space. This scoring function promotes tighter packing and reduces fragmentation. Jobs are deployed as Kubernetes pods using a custom Docker image, with resource requests directly set to their measured CPU and memory usage. If a job cannot fit on any currently active node, an additional node is activated and added to the placement pool.

After all pods are deployed, the system computes per-node utilization and residual imbalance. Fragmentation is quantified using the Resource Fragmentation Index (RFI) defined later in Section II-F. Node level energy consumption is then estimated using the linear utilization model in (1). Nodes operating below 35% of both CPU and memory utilization are flagged as safe shutdown candidates, allowing the system to identify idle machines that can be powered off without violating placement feasibility or cluster stability.

Algorithm 2: Greedy Consolidation for FAS.

Require: Set of machines $\mathcal{M}$, consolidation rounds R
Ensure: Number of machines freed ($\text{Freed}_{\text{Count}}$)

$\text{Freed}_{\text{Count}} \leftarrow 0$
for $r = 1$ to R **do**
  Sources $\leftarrow$ machines with jobs, sorted by increasing utilization
  $\text{Machines}_{\text{freed}} \leftarrow 0$
  for each source machine $M_s \in$ Sources **do**
    $\text{Jobs}_{\text{move}} \leftarrow$ jobs currently on M_s
    tentative_placements $\leftarrow \emptyset$
    all_moved $\leftarrow$ True
    for each job $j \in \text{Jobs}_{\text{move}}$ **do**
      Find first $M_d \in \mathcal{M}, M_d \neq M_s$, such that M_d.can_fit(j)
      if M_d found **then**
        M_d.place(j) {Tentative placement}
        tentative_placements.add(M_d, j)
      else
        {Rollback all tentative placements}
        rollback tentative_placements
        all_moved $\leftarrow$ False
        break
      end if
    end for
    if all_moved **then**
      M_s.clear_all_jobs() {Source machine is now empty}
      $\text{Freed}_{\text{Count}} \leftarrow \text{Freed}_{\text{Count}} + 1$
      $\text{Machines}_{\text{freed}} \leftarrow 1$
    end if
  end for
  if $\text{Machines}_{\text{freed}} = 0$ **then**
    break
  end if
end for
return $\text{Freed}_{\text{Count}}$

F. WORKLOAD MODEL

The scheduling system operates on a multi-resource workload consisting of CPU memory demands derived from the Alibaba Cluster Trace. Each incoming job is modeled as a three-dimensional tuple

$$j_i = (c_i,\ m_i,\ d_i)$$

where c_i and m_i denote the CPU and memory requirements, and d_i is the execution duration. Jobs are further classified into short and long categories as

$$\text{is_short}(j_i) = \begin{cases} 1, & d_i \leq \text{median}(d), \\ 0, & \text{otherwise.} \end{cases}$$

Each machine in the cluster is modeled as

$$M_k = (C_k,\ Mem_k),$$

where C_k and M_k denote the total CPU and memory capacity, respectively. After scheduling a set of jobs onto machine M_k, the remaining fractional resources are computed as

$$f_{c,k} = \frac{C_k - \sum_{j_i \in M_k} c_i}{C_k}, \qquad f_{m,k} = \frac{M_k - \sum_{j_i \in M_k} m_i}{M_k}.$$

To measure imbalance in the remaining resources, the Resource Fragmentation Index (RFI) is defined as

$$\text{RFI}_k = 1 - \frac{\min(f_{c,k},\ f_{m,k})}{\max(f_{c,k},\ f_{m,k})}.$$

The objective of scheduling is therefore to select a machine M_k for each job j_i such that post-placement fragmentation is minimized. To determine the most suitable placement node, each feasible machine is evaluated using the following scoring function:

$$S_k(j_i) = \text{RFI}_k^{(\text{after})} + w_i \left(f_{c,k} + f_{m,k}\right),$$

where $w_i = 2.0$ for short jobs (to encourage tight packing) and $w_i = 0.5$ for long jobs (to preserve larger blocks of resources). The machine with the minimum score is selected for placement.

This workload formulation enables the system to reason about multi-resource imbalance, classify jobs based on temporal behavior, and compute the expected fragmentation resulting from each scheduling decision.

G. ENERGY AWARE NODE CONSOLIDATION

Energy aware node consolidation is performed on the live Kubernetes cluster by reconstructing node utilization from the resource requests of all `fas-job` pods. Each node's CPU and memory usage is computed from Kubernetes API reports, after which nodes are categorized into empty nodes eligible for immediate shutdown, low utilization nodes to be drained, and high-utilization nodes that remain active. The consolidation logic selects the least loaded nodes as drain targets while preserving the two most utilized nodes as the active consolidation anchors.

For pods residing on nodes selected for draining, a simulated repacking step evaluates whether each pod can be migrated onto the target nodes using a combined DRF and fragmentation based score to ensure capacity feasibility. Nodes are marked as removable only if all pods can be reassigned without violating resource constraints. Cluster energy consumption is then estimated before and after consolidation using the utilization based power function in (1), assuming powered-off nodes draw zero power. When consolidation is executed, removable nodes are cordoned and annotated as powered off, and their pods are deleted and recreated on the designated target nodes, completing the energy aware consolidation workflow. The current consolidation model assumes negligible migration overhead during pod relocation. In the present implementation, migration operations are assumed to incur negligible delay due to lightweight container relocation, and their impact is reported using the MigrationOps metric

in Section V. Future enhancements will incorporate migration cost estimation to better capture real-world operational latency.

H. EXPERIMENTAL EVALUATION

The final empirical evaluation leverages real execution metrics collected from a Kubernetes deployment, together with detailed placement records, to enable data driven and intelligent performance assessment. Node level summaries are analyzed to compute CPU and memory utilization across the cluster, from which overall utilization trends and residual resource fractions are derived. Cluster wide fragmentation is quantified using the Resource Fragmentation Index (RFI), computed from the ratio of minimum-to-maximum leftover CPU memory fractions per node, providing a fine grained measure of resource imbalance. Energy consumption is estimated using the linear utilization power relationship in (1), yielding both the full power energy footprint with all nodes active and the reduced energy consumption achievable through intelligent consolidation and automated node power management.

The resulting energy savings demonstrate the effectiveness of the consolidation strategy. Fragmentation statistics from earlier placement experiments are incorporated for comparative analysis. A summary table compares the proposed scheduler with baseline approaches in terms of utilization, fragmentation reduction, energy savings, and SLA violation rate. Overall, the proposed framework improves resource utilization and reduces energy consumption in large-scale containerized cloud environments.

IV. EXPERIMENTAL SETUP AND EVALUATION MEASURES

All experiments were conducted using Python-based simulation and a five-node Kubernetes cluster. Workload patterns and machine capacities were derived from a public cluster trace dataset and deployed as containerized Kubernetes pods using the Python client API.

Step 1. Dataset Processing and Workload Extraction:

- Machine configurations, CPU memory capacities, and workload traces have been extracted from the cluster trace dataset.
- Job attributes such as CPU demand, memory demand, and execution duration have been profiled to generate representative scheduling workloads.

Step 2. Workload Generation and Classification:

- Synthetic workloads have been produced using lognormal sampling to preserve real world variability.
- Jobs have been classified as short or long tasks based on their execution duration characteristics.

Step 3. Simulation Environment Setup:

- A Python-based simulator has been developed to emulate cluster nodes, job arrivals, and scheduling decisions.
- The four schedulers—FF, BFD, DRF, and the proposed FAS have been implemented within a uniform evaluation framework.
- Consolidation routines have been executed after the initial placement to identify machines eligible for shutdown.

Step 4. Kubernetes Deployment and Real Execution:

- The generated workloads have been containerized and deployed as Kubernetes pods using the official Python API.
- A five-node cluster has been used to observe real scheduling behaviour, resource utilization patterns, and node idling events.
- Node level power consumption has been estimated using a linear utilization power model.

Step 5. Evaluation Metrics: The following standard scheduling and consolidation metrics have been used:

- *Cluster Utilization:* Average CPU and memory utilization across active nodes.
- *Resource Fragmentation Index (RFI):* Quantifies imbalance in post-placement free resource fractions.
- *Energy Consumption:* Energy consumption is computed using the power model defined in (1).
- *Energy Savings:* Difference between energy with all nodes active and energy after powering off underutilized nodes.
- *Job Acceptance Rate:* Number of successfully placed jobs.
- *Nodes Freed:* Count of machines powered off through consolidation.
- *SLA Violation Rate:* Fraction of jobs whose resource guarantees or deadlines have not been met.

These steps collectively establish a unified experimental framework for assessing the effectiveness of the proposed FAS method in improving utilization, reducing fragmentation, and lowering energy consumption relative to baseline schedulers.

V. RESULTS

A comprehensive evaluation of the proposed Idle Fragmentation-Aware Resource Scheduling framework has been conducted using the Alibaba Cluster Trace dataset, simulation based scheduling, and a multi node Kubernetes deployment. The results have collectively demonstrated improvements in utilization, fragmentation behaviour, energy efficiency, and consolidation effectiveness.

The proposed scheduling mechanisms are evaluated using four core quantitative metrics, namely: (i) Average Utilization (AU), which measures effective CPU memory packing efficiency across machines; (ii) Resource Fragmentation Index (RFI), which quantifies imbalance in idle CPU memory regions; (iii) Energy Consumption (EC), computed using the utilization dependent power model defined in Section III; and (iv) Job Acceptance Rate (JAR), which indicates placement feasibility without fragmentation induced rejections. The overall workflow of the proposed Fragmentation-Aware Scheduling Framework is illustarted in Fig.3.

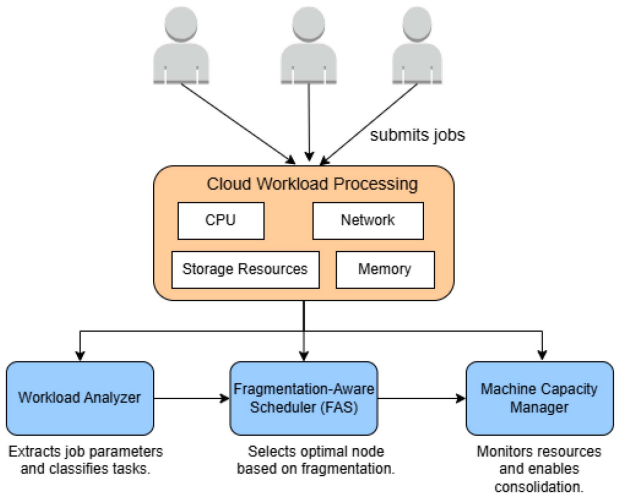


FIGURE 3. Proposed Fragmentation-Aware task scheduling Framework.

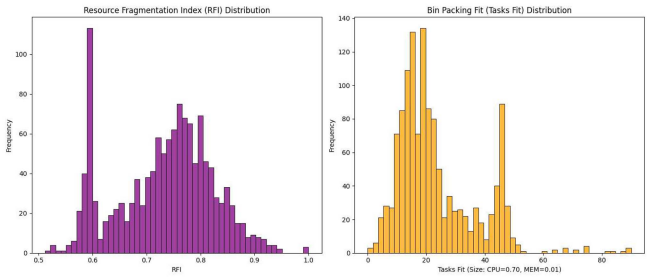


FIGURE 4. RFI distribution and task-fit capacity.

A. FRAGMENTATION AND RESOURCE PROFILING

The Alibaba dataset has been processed to produce 187,962 machine–usage entries corresponding to 1,310 machines, consistent with prior workload characterization studies using the Alibaba Cluster Trace [14]. CPU values have been normalized to core equivalent units, and memory usage has been expressed as a fraction of node capacity. The 75th percentile of real batch workloads has yielded a representative task size of 0.70 CPU cores and 0.0085 normalized memory units.

Fig. 4 has illustrated the distribution of the Resource Fragmentation Index (RFI) and the number of tasks each machine has been able to host. The figure shows that most machines exhibit RFI values between 0.6 and 0.8, indicating that CPU and memory resources are not left idle in balanced proportions. On average, each machine is capable of fitting approximately 23.33 tasks; however, fragmentation restricts the number of practically allocatable slots.

To further clarify the interpretation of RFI values, consider an example where a machine has 40% free CPU and 10% free memory after placement. The Resource Fragmentation Index is computed as:

$$\text{RFI} = 1 - \frac{\min(0.40,\ 0.10)}{\max(0.40,\ 0.10)} = 1 - \frac{0.10}{0.40} = 0.75$$

TABLE 4. Baseline First-Fit Energy Comparison With Ideal Packing Configuration

Metric	Value
Machines (Total / Ideal)	1310 / 684
Machines Wasted	626
Energy (Baseline FF)	2839.67 kWh
Energy (Ideal)	1712.87 kWh
Wasted Energy	1126.80 kWh

TABLE 5. Comparison of Scheduling Algorithms

Scheduler	Jobs Placed	RFI	Nodes Off	Migration Ops	Energy (kWh)
FF	4877	0.9617	67	0	3709.1
BFD	4877	0.9676	82	0	3682.1
DRF	4583	0.8241	0	0	3598.4
FAS (Proposed)	4877	0.9646	59	2475	3723.5

This value indicates a high level of imbalance, meaning that although free resources exist, they cannot be effectively utilized for additional workload placement.

B. BASELINE SCHEDULING AND ENERGY INEFFICIENCY

A First-Fit scheduler has been tested using 5,000 identical incoming jobs. Although all jobs have been successfully placed, the allocation has resulted in substantial unused capacity scattered across machines, with a measured RFI of 0.7756. Despite the presence of 1,310 machines, an ideal configuration would require only 684 machines, meaning that 626 machines have remained unnecessarily active.

Table 4 has reported the numerical comparison between the baseline First-Fit allocation and an ideal packing configuration. The results have shown that 1.126 MWh of energy have been wasted due to excessive node activation, highlighting the need for fragmentation-aware placement.

C. COMPARISON OF SCHEDULING ALGORITHMS

Four algorithms—FF, BFD, DRF, and the proposed FAS—have been evaluated. FF, BFD, and FAS have each placed 4,877 jobs, whereas DRF has rejected 417 jobs due to fairness-driven placement. Table 5 has summarised the complete set of metrics. The proposed FAS scheduler performs a significant number of consolidation operations, reflected by 2475 migration actions executed during the post-placement consolidation phase. These migration operations enable redistribution of workloads from lightly utilized nodes to more densely packed nodes, allowing 59 nodes to be safely powered off. This behavior demonstrates that FAS improves consolidation effectiveness without reducing job placement success.

The experimental results show that the DRF scheduler achieved the lowest fragmentation index (0.8241), indicating the most balanced resource allocation among the evaluated methods in terms of fairness-driven placement. However, the proposed Fragmentation-Aware Scheduler (FAS) demonstrates competitive fragmentation behavior (0.9646) while

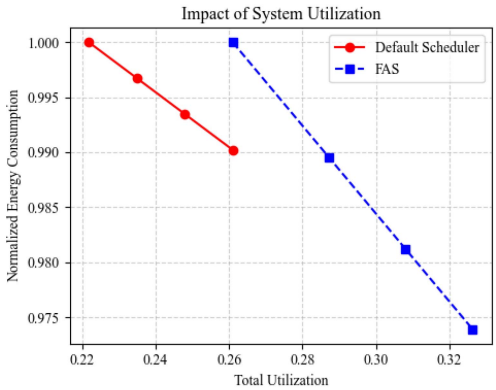


FIGURE 5. Impact of system utilization on normalized energy consumption.

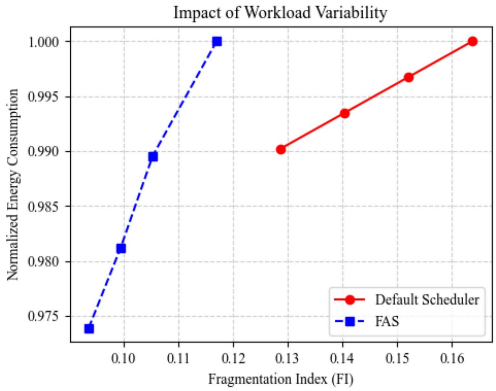


FIGURE 6. Impact of workload variability (fragmentation index) on normalized energy consumption.

significantly improving consolidation effectiveness and enabling energy savings through node shutdown. Unlike DRF, which prioritizes fairness and resource balancing, FAS jointly optimizes resource packing and power efficiency, allowing inactive nodes to be powered off and thereby reducing overall energy consumption.

Fragmentation values reported in this stage correspond to pre-consolidation placement behavior, whereas the final end-to-end evaluation reflects post-consolidation improvements achieved through the proposed FAS workflow.

D. CONTAINERIZED DEPLOYMENT IN KUBERNETES

To validate the scheduling strategy in an operational environment, 120 containerized workloads have been deployed across a five-node Kubernetes cluster. The proposed scheduler has consolidated all pods onto three nodes, leaving two nodes completely idle and eligible for shutdown. Large-scale simulation experiments were conducted using 1,310 machines derived from the Alibaba dataset, while real-world deployment validation was performed on a five-node Kubernetes cluster.

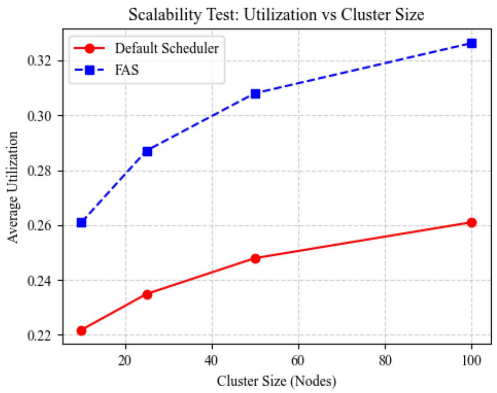


FIGURE 7. Scalability evaluation: utilization versus cluster size.

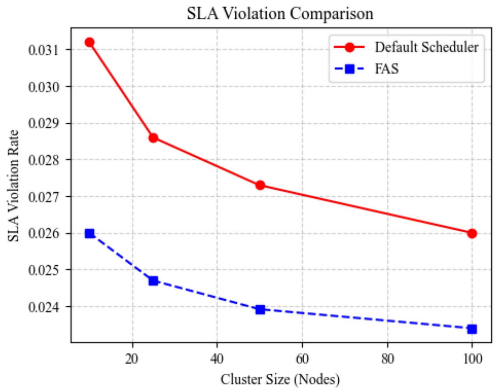


FIGURE 8. SLA violation rate comparison across cluster sizes.

TABLE 6. Energy Usage in Kubernetes Deployment

Metric	Value (kWh)
Energy (All Nodes Active)	9.77
Energy (After Power-Off)	6.17
Energy Saved	3.60

The evaluation trends are illustrated through four key sensitivity and scalability plots (Figs. 5–8). Fig. 5 presents the impact of increasing system utilization on normalized energy consumption, where FAS consistently consumes less energy than the default scheduler due to tighter packing and reduced idle capacity. Fig. 6 shows the relationship between fragmentation index and energy usage, demonstrating that lower fragmentation achieved by FAS directly corresponds to improved energy efficiency. Fig. 7 evaluates scalability across cluster sizes ranging from 10 to 100 nodes, indicating that FAS maintains higher utilization as the cluster grows, while the baseline scheduler experiences greater inefficiencies. Fig. 8 shows that FAS preserves SLA stability even under higher packing efficiency, maintaining violation rates below 3% across all cluster sizes.

Table 6 has shown that energy usage has decreased from 9.77 kWh to 6.17 kWh after powering off unused nodes, amounting to a 36.8% reduction.

TABLE 7. Node Level Power Measurements

Node	Before (W)	After (W)
minikube	171.33	0.00
minikube-m02	171.33	182.00
minikube-m03	171.33	182.00
minikube-m04	150.00	0.00
minikube-m05	150.00	0.00

TABLE 8. Comparison of Baseline Vs. FAS Scheduler

Metric	Baseline	FAS (Proposed)
Average Utilization (↑)	0.209	0.261
Fragmentation Index (↓)	0.175	0.117
Energy Savings (↑)	0%	36.86%
SLA Violations (↓)	0.031	0.026
Job Acceptance Rate	100%	100%

As shown in Fig. 7, utilization increases from approximately 0.70 at 10 nodes to 0.86 at 100 nodes, demonstrating effective scalability of the proposed FAS scheduler.

Moreover, the fragmentation index (RFI) has been shown to remain low across all tested cluster sizes, which has enabled more nodes to be powered off and consequently reduced the overall energy consumption per unit of work. SLA violation rates have been maintained below the acceptable threshold (3%) as cluster size has grown, which demonstrates that higher packing efficiency has not led to service instability.

E. ENERGY AWARE NODE CONSOLIDATION

A consolidation stage has been executed to drain underutilized nodes. Three nodes have been freed and marked for shutdown. Table 7 has reported the measured per-node power before and after consolidation, showing a reduction from 813.99 W to 364.00 W (55.3%).

F. END-TO-END EVALUATION

Table 8 has presented the complete comparison between baseline Kubernetes scheduling and the proposed FAS. FAS has reduced the fragmentation index from 0.175 to 0.117, increased utilization, improved energy efficiency by 36.86%, and preserved 100% job acceptance.

F. HIGHLIGHTS OF EXPERIMENTAL RESULTS

- The proposed FAS framework reduced the fragmentation index by **33.14%** (from 0.175 to 0.117), resulting in more balanced CPU and memory utilization across cluster nodes.
- Average cluster utilization improved by **24.9%** (from 0.209 to 0.261), demonstrating tighter workload packing and reduced idle resource capacity.
- Energy consumption in the Kubernetes deployment decreased by approximately **36.9%** (9.77 kWh → 6.17 kWh) through fragmentation-aware consolidation and node shutdown.
- The proposed scheduler maintained **100%** job acceptance while keeping SLA violation rates low (0.031 → 0.026), indicating reliable workload execution under higher packing efficiency.
- During consolidation, the FAS scheduler performed **2,475 migration operations**, enabling safe redistribution of workloads and improved node consolidation efficiency.

VI. CONCLUSION

This study proposes an Idle Fragmentation-Aware Scheduling (FAS) framework to address scattered CPU–memory resources in large-scale Kubernetes clusters. By quantifying residual imbalance using the Resource Fragmentation Index (RFI) and incorporating fragmentation-aware placement, FAS improves workload packing and consolidation efficiency. Experimental evaluation using Alibaba trace-driven workloads and Kubernetes deployment shows that the proposed approach reduces fragmentation by 33.14%, improves average cluster utilization by 24.9%, and achieves up to 36.9% energy savings through safe node shutdown, while maintaining 100% job acceptance and low SLA violation rates.

Despite these improvements, certain limitations remain. The current framework does not explicitly model migration overhead costs, workload uncertainty, or non-linear thermal power characteristics, and the live validation is performed on a limited-scale Kubernetes cluster.

Future work will extend the FAS framework by incorporating migration-aware cost modeling, predictive workload estimation, and non-linear energy models to better reflect real-world dynamics. Additional large-scale and multi-cluster deployments will also be explored to enhance scalability and practical applicability. Overall, this work demonstrates that fragmentation-aware scheduling is a practical approach for improving resource utilization and energy efficiency in containerized cloud environments.

ACKNOWLEDGMENT

The authors gratefully acknowledge Sri Mata Amritanandamayi Devi (Amma), Chancellor, Amrita Vishwa Vidyapeetham, for her inspiration and for providing financial support for the Article Processing Charges (APC) of this publication.

REFERENCES

[1] Y. Chen, Z. Zhang, Y. Deng, G. Min, and L. Cui, "A combined trend virtual machine consolidation strategy for cloud data centers," *IEEE Trans. Comput.*, vol. 73, no. 9, pp. 2150–2164, Sep. 2024.

[2] K. Senjab, S. Abbas, N. Ahmed, and A. U. R. Khan, "A survey of Kubernetes scheduling algorithms," *J. Cloud Comput.*, vol. 12, no. 1, 2023, Art. no. 87.

[3] W. Rao and H. Li, "Energy-aware scheduling algorithm for microservices in Kubernetes clouds," *J. Grid Comput.*, vol. 23, no. 1, 2025, Art. no. 2.

[4] H. Li, W. Rao, B. Hu, Y. Tian, and J. Shen, "Energy-aware elastic scaling algorithm for microservices in Kubernetes clouds," *J. Netw. Comput. Appl.*, vol. 242, 2025, Art. no. 104218. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1084804525001158

[5] A. Daimari, M. Jansen, and D. Bonetta, "Energy consumption of heuristic Kubernetes schedulers," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Soft.*, 2025, pp. 1–7. [Online]. Available: https://atlarge-research.com/pdfs/2025-adaimari-isp.pdf

[6] A. Pashaeehir, S. Shariati, S. Shafaghi, M. Moghimi, and M. Momtazpour, "KubedSM: A Kubernetes-based dynamic scheduling and migration framework for cloud-assisted edge clusters," 2025, *arXiv:2501.07130*.
[7] F. Kiaee and E. Arianyan, "Joint VM and container consolidation with auto-encoder based contribution extraction of decision criteria in edge-cloud environment," *J. Netw. Comput. Appl.*, vol. 233, 2025, Art. no. 104049.
[8] N. Pinnapareddy, "Carbon conscious scheduling in Kubernetes to cut energy use and emissions," *Int. J. Comput. Exp. Sci. Eng.*, vol. 11, pp. 2025, doi: 10.22399/ijcesen.3785.
[9] W. Yao, Z. Wang, Y. Hou, X. Zhu, X. Li, and Y. Xia, "An energy-efficient load balance strategy based on virtual machine consolidation in cloud environment," *Future Gener. Comput. Syst.*, vol. 146, pp. 222–233, 2023.
[10] A. Gopu et al., "Energy-efficient virtual machine placement in distributed cloud using NSGA-III algorithm," *J. Cloud Comput.*, vol. 12, no. 1, 2023, Art. no. 124.
[11] A. U. Rehman et al., "EEVMC: An energy efficient virtual machine consolidation approach for cloud data centers," *IEEE Access*, vol. 12, pp. 105234–105245, 2024.
[12] L. Mao, R. Chen, H. Cheng, W. Lin, B. Liu, and J. Z. Wang, "A resource scheduling method for cloud data centers based on thermal management," *J. Cloud Comput.*, vol. 12, no. 1, 2023, Art. no. 84.
[13] J. Lin, W. Lin, W. Wu, W. Lin, and K. Li, "Energy-aware virtual machine placement based on a holistic thermal model for cloud data centers," *Future Gener. Comput. Syst.*, vol. 161, pp. 302–314, 2024.
[14] X. Li, L. Pan, and S. Liu, "A DRL-based online VM scheduler for cost optimization in cloud brokers," *World Wide Web*, vol. 26, no. 5, pp. 2399–2425, 2023.
[15] P. Li and J. Cao, "Virtual machine consolidation with multi-step prediction and affinity-aware technique for energy-efficient cloud data centers," *Comput., Mater. Continua*, vol. 76, no. 1, pp. 81–105, 2023.
[16] P. Choppara and M. Sudheer, "An efficient deep reinforcement learning based task scheduler in cloud-fog environment," *Cluster Comput.*, vol. 28, pp. 2024, Art. no. 67.
[17] C. Centofanti, J. Santos, V. Gudepu, and K. Kondepu, "Impact of power consumption in containerized clouds: A comprehensive analysis of open-source power measurement tools," *Comput. Netw.*, vol. 245, 2024, Art. no. 110371.
[18] A. Gudkov, P. Popov, and S. Romanov, "Balcon—Resource balancing algorithm for VM consolidation," *Future Gener. Comput. Syst.*, vol. 147, pp. 265–274, 2023.
[19] B. M. Beena, P. C. Ranga, T. S. S. Manideep, S. Saragadam, and G. Karthik, "A green cloud-based framework for energy-efficient task scheduling using carbon intensity data for heterogeneous cloud servers," *IEEE Access*, vol. 13, pp. 73916–73938, 2025. [Online]. Available: https://api.semanticscholar.org/CorpusID:277986408
[20] B. M. Beena, P. Cheluvasai Ranga, A. Vinitha Chowdary, R. Gamidi, M. Hemasri, and T. Muppala, "Green video transcoding in cloud environments using Kubernetes: A framework with dynamic renewable energy allocation and priority scheduling," *IEEE Access*, vol. 13, pp. 116362–116390, Jan. 2025.
[21] H. Dong et al., "Towards performance and energy aware kubernetes scheduler," *ACM SIGEnergy Energy Inform. Rev.*, vol. 5, pp. 69–75, 2025.
[22] T. S. Reddy and B. B. M, "Analysis of green computing models on AWS using machine learning algorithms," in *Proc. 1st Int. Conf. Women Comput.*, 2024, pp. 1–6.
[23] T. Menouer, "Kcss: Kubernetes container scheduling strategy," *J. Supercomputing*, vol. 77, no. 4, pp. 4267–4293, 2021.
[24] M. López-Rauhut, E. Stafford, and J. Bosque, "Intelligent energy pairing scheduler (INEPS) for heterogeneous HPC clusters," *J. Supercomputing*, vol. 81, 2025, Art. no. 427.
[25] N. Gogineni and S. M. Sivalingam, "A systematic review on recent methods of scheduling and load balancing for containers in distributed environments," *Int. J. Adv. Technol. Eng. Exploration*, vol. 11, no. 116, 2024.
[26] V. Dakić, M. Kovač, and J. Slovinac, "Evolving high-performance computing data centers with Kubernetes, performance analysis, and dynamic workload placement based on machine learning scheduling," *Electronics*, vol. 13, no. 13, 2024, Art. no. 2651. [Online]. Available: https://www.mdpi.com/2079-9292/13/13/2651
[27] D. Liu, Y. Xia, C. Shan, K. Tian, and Y. Zhan, "A Kubernetes-based scheme for efficient resource allocation in containerized workflow," *Future Gener. Comput. Syst.*, vol. 166, 2025, Art. no. 107699.
[28] A. Chaves et al., "Neuroscaler: Towards energy-optimal autoscaling for container-based services," 2026, *arXiv:2602.08191*.
[29] J. Yan et al., "Solving the batch stochastic bin packing problem in cloud: A chance-constrained optimization approach," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 2169–2179.
[30] B. Beena, D. Madhusoodanan, N. Sagar, R. Vismaya, and H. Santhosh, "Optimizing CPU power consumption for sustainable cloud operations," in *Proc. Int. Conf. ICT Sustain. Develop.*, 2025, pp. 83–92.
[31] S. Tiwari and B. B. M. , "A neighborhood inspired multiverse scheduler for energy and makespan optimized task scheduling for green cloud computing systems," *IEEE Access*, vol. 12, pp. 157272–157298, 2024.
[32] T. Deepika and N. Dhanya, "Multi-objective prediction-based optimization of power consumption for cloud data centers," *Arabian J. Sci. Eng.*, vol. 48, no. 2, pp. 1173–1191, 2023.
[33] N. Chaurasia, M. Kumar, R. Chaudhry, and O. P. Verma, "Comprehensive survey on energy-aware server consolidation techniques in cloud computing," *J. Supercomputing*, vol. 77, no. 10, pp. 11682–11737, 2021.
[34] G. Schryen, "A unifying theory of parallel performance: Asymptotic speedup, efficiency, and scalability typologies," *J. Parallel Distrib. Comput.*, vol. 181, pp. 119–134, 2024.
[35] A. Baydoun and A. S. Zekri, "Towards energy-efficient cloud computing: A review of network-aware VM placement approaches," *J. Inf. Syst. Telecommunication*, vol. 3, no. 51, 2025, Art. no. 210.
[36] S. N. Nayini, V. Gade, M. P. R. Gudapareddy, V. Peddineni, and B. M. Beena, "Revolutionizing healthcare: Cloud-based health information exchange and disease prediction," in *Proc. Symp. Comput. Intell. Syst.*, vol. 3682, 2024. [Online]. Available: https://ceur-ws.org/Vol-3682/Paper9.pdf
[37] B. M. Beena, A. Kothari, V. Nikhil, N. Rohit, and C. S. R. Prashanth, "Efficient unified self-service sustainable cloud portal with cloud connectors for multi-platform orchestration," *Res. Square*, Mar. 2025, doi: 10.21203/rs.3.rs-6218612/v1.
[38] C. S. Kumar, N. S. Jaswanth, V. D. Reddy, P. S. Reddy, and B. M. Beena, "Cloud-based image processing and image augmentation," in *Proc. Int. Conf. Signal Process. Integrated Netw.*, S. H. Gupta, A. Amphawan, H. Q. Ngo, and M. Lopez-Benitez, Eds., Singapore, 2025, pp. 169–186.
[39] "Alibaba cluster trace dataset V2017," 2017. [Online]. Available: https://github.com/alibaba/clusterdata/tree/master/cluster-trace-v2017



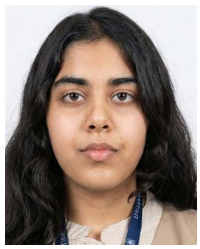
B.M. BEENA (Senior Member, IEEE) received the B.E. degree in computer science and engineering from the PDA College of Engineering, Gulbarga University, Gulbarga, Karnataka, India, the M.Tech. degree in computer science and engineering from the BMS College of Engineering, Bengaluru, India, and the Ph.D. degree from Visvesvaraya Technological University (VTU), Belagavi, Karnataka, India. She was with VTU-affiliated Engineering Colleges in Bengaluru and RUBIXE and Wells Fargo, Bengaluru, as a Senior Consultant, for five years. She has more than 27 years of Academic and Industry experience and was a Senior Consultant with Wells Fargo, Bengaluru office, India. She is currently an Associate Professor with the Department of Computer Science and Engineering, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Bengaluru. She is also guiding six Ph.D. degree Scholars in the green cloud computing and blockchain technology domain. Her research interests include green cloud computing, blockchain technology, and artificial intelligence.



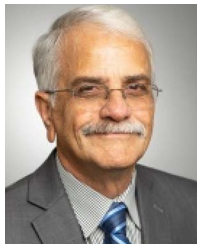
JIYA BORIKAR is currently working toward the B.Tech. degree in computer science and engineering (with a specialization in artificial intelligence) with Amrita Vishwa Vidyapeetham, Bengaluru, India. Her research interests include the intersection of machine learning, deep learning, and cloud computing, with a focus on energy-efficient and sustainable computing solutions. She has contributed to several real world projects involving machine learning, cloud services, and IoT-based intelligent systems. Her research interests include green cloud scheduling, applied AI systems, and scalable web technologies, and is also passionate about building practical, problem-solving applications, having a strong interest in modern web development, including the design of interactive and user centric platforms. .



LOHITHA KANISETTYPALLI is currently working toward the B.Tech. degree in computer science and engineering (with a specialization in artificial intelligence) with Amrita Vishwa Vidyapeetham, Bengaluru, India. She has contributed to several projects in the fields of machine learning, deep learning, cloud computing, IoT, and real-world AI applications. Her interests include machine learning, deep learning, cloud computing and modern web development, and is also passionate about building scalable and problem-driven systems, with interests including green computing, sustainable cloud scheduling, applied deep learning, and intelligent web technologies.



HARSDA SHRIVASTAVA is currently working toward the B.Tech. degree with a specialization in artificial intelligence. She has hands-on experience working with AWS, containerization, and Kubernetes, along with applying deep learning and NLP to biomedical datasets. She aspires to contribute, as a Cloud Engineer or AI/ML Engineer in developing high-performance, data-driven, and scalable applications. Her research interests include cloud computing, DevOps, and full-stack development (MERN), with a strong focus on building scalable and secure cloud-based systems.



RAMESH RAMASWAMY is currently a Senior Professor in the fields of conceptual modeling, database systems, decision analysis, and IT economics. He has held leadership roles in both Academia and Business, developing Dual-Degree M.S./M.B.A. Programs, and Conducting IT Training Programs Globally. His research interests include health information interchange, service contracts SLAs, and cloud computing predictive analytics. His work, which has been funded by the NSF, Google, Samsung, and U.S. Military Research Programs, uses stochastic optimization, mathematical programming, and predictive modeling. He Co-Founded *Information Systems Frontiers* and is also a Co-Editor-in-Chief and Associate Editor for the *INFORMS Journal on Computing*. His numerous articles have appeared in the top IEEE Journals, such as IEEE TRANSACTIONS COMPUTERS AND MANAGEMENT SCIENCE.